Brainstore
GROUP

GUIDE D'INSTALLATION

ERP Transport & Voyages

Comment installer et démarrer l'application, pas à pas — même sans expérience en informatique.

Serveur central en ligne

Serveurs d'agence (Linux)

Synchronisation automatique

Version 1.0 · Septembre 2026 · À lire avant toute installation

Sommaire

1. À qui s'adresse ce guide & comment le lire
2. Comprendre comment l'application fonctionne
3. Le vocabulaire, en mots simples
4. Installation du serveur central (en ligne)
5. Installation dans une agence (serveur Linux)
6. Les réglages (variables d'environnement) expliqués
7. Première utilisation
8. Sauvegardes, coupures de courant & entretien
9. En cas de problème (dépannage)

---

1 À qui s'adresse ce guide

Ce guide vous accompagne du début à la fin. Vous n'avez pas besoin d'être informaticien : il suffit de **lire chaque étape dans l'ordre** et de recopier exactement les commandes encadrées.

> **Comment lire les commandes**
> Les blocs sur fond sombre sont des **commandes** à taper (ou coller) dans le « Terminal » du serveur. Recopiez-les **exactement**, puis appuyez sur **Entrée**. Les lignes qui commencent par `#` sont des **commentaires** : elles expliquent, elles ne s'exécutent pas.

> **Deux installations, deux publics**
> Le **serveur central** (partie 4) est installé **une seule fois**, en général par votre prestataire technique. L'installation **en agence** (partie 5) est celle que vous ferez le plus souvent — elle est détaillée pas à pas.

> **Avant de commencer, munissez-vous de :**
> - ✓ l'adresse du serveur central (fournie par le prestataire), ex. `https://erp.mondomaine.com`
> - ✓ le **jeton de synchronisation** (SYNC_TOKEN) — un mot de passe partagé
> - ✓ l'**identifiant de l'agence** (un code type UUID) créé au préalable sur le serveur central
> - ✓ un serveur (ordinateur) sous **Linux** allumé en permanence dans l'agence

2 Comment l'application fonctionne

L'application est organisée en « étoile » : un **serveur central** en ligne, et un **serveur dans chaque agence**. Les agences travaillent même sans Internet et se synchronisent automatiquement avec le centre dès que la connexion revient.

Élément	Rôle, en une phrase
Serveur central (en ligne)	La « référence » : il détient la configuration (institutions, agences, tarifs, comptes) et gère la **réservation en ligne** (site web + WhatsApp).
Serveur d'agence (local)	Fait fonctionner l'agence au quotidien (ventes au guichet), **même sans Internet**. Il envoie ses ventes au centre et reçoit la configuration du centre.
Synchronisation	Un programme automatique qui, toutes les 1–2 minutes, fait « monter » les ventes vers le centre et « descendre » la configuration.

À retenir

La configuration (tarifs, agences, utilisateurs…) se modifie **sur le serveur central**. Elle redescend ensuite automatiquement dans les agences (où elle est en lecture seule).

Une seule agence ?

Vous pouvez tout faire tourner sur le serveur central uniquement : dans ce cas, mettez le réglage `SYNC_ENABLED` à `0` et ignorez la partie 5.

3 Le vocabulaire, en mots simples

Serveur
Un ordinateur qui reste allumé en permanence et fait tourner l'application.

Terminal
Une fenêtre où l'on tape des commandes texte (au lieu de cliquer). C'est là que vous collerez les commandes de ce guide.

Linux
Le système d'exploitation du serveur d'agence (comme Windows ou macOS, mais pour serveurs). Nous recommandons **Ubuntu**.

Docker
Un outil qui installe et lance l'application « clé en main », sans avoir à installer chaque composant à la main.

Base de données (PostgreSQL)
Là où sont enregistrées toutes les données (réservations, colis, caisses…).

Variable d'environnement
Un réglage (une ligne « nom = valeur ») que l'on met dans un fichier `.env` pour configurer l'application.

UUID
Un identifiant unique, long, du type `3f2a…-…-9c1b`. On en génère un pour l'agence.

4 Installation du serveur central (en ligne)

Le serveur central se compose de deux parties hébergées dans le cloud : le **backend** (le moteur + la base de données) sur **Railway**, et le **frontend** (l'interface web) sur **Vercel**. La mise en ligne est automatique grâce à un « pipeline » (GitHub Actions).

> **Partie plus technique**
> Cette étape est généralement réalisée **une fois** par votre prestataire. Vous pouvez la survoler et passer à la partie 5 si le serveur central est déjà en place.

4.1 Le backend (Railway)

1. Créez un projet Railway à partir du dépôt de code, puis réglez **Root Directory =** `backend`.
2. Ajoutez une base **PostgreSQL** (Railway fournit alors automatiquement `DATABASE_URL`).
3. Renseignez les variables (voir partie 6) :
   `DJANGO_SETTINGS_MODULE=config.settings.production`, `SECRET_KEY`,
   `CORS_ALLOWED_ORIGINS`, `CSRF_TRUSTED_ORIGINS`.
4. Notez l'adresse publique du backend, ex. `https://brainstore-production.up.railway.app`.

> **Bon à savoir**
> Au démarrage, le backend applique tout seul les mises à jour de la base (« migrations »), et l'interface web est servie automatiquement. Rien à faire manuellement.

4.2 Le frontend (Vercel)

1. Créez un projet Vercel à partir du même dépôt, avec **Root Directory =** `angular`.
2. Dans le fichier `angular/src/environments/environment.prod.ts`, indiquez l'adresse du backend suivie de `/api/v1`, et l'identifiant de l'institution servie par la réservation en ligne :

```
# exemple
apiUrl: 'https://<backend-railway>/api/v1',
onlineCompanyId: '<UUID de l'institution>',
```

3. Une fois le frontend en ligne, récupérez son adresse `*.vercel.app`.
4. Reportez cette adresse dans `CORS_ALLOWED_ORIGINS` et `CSRF_TRUSTED_ORIGINS` côté Railway.

> **Créer le premier compte**
> Dans la console du backend, créez le super-administrateur :

```
python manage.py createsuperuser
```

Puis, connecté dans l'interface : **Super Admin** → **Institutions** → **Nouvelle institution**. En saisissant l'e-mail du premier administrateur, l'application crée automatiquement l'**agence Siège** et son compte (un mot de passe temporaire s'affiche une seule fois).

5 Installation dans une agence (serveur Linux)

C'est l'installation la plus courante. Suivez les étapes **dans l'ordre**. Comptez environ 30 minutes la première fois.

5.1 Installer Docker (une seule fois)

Ouvrez le Terminal du serveur et collez :

```
# Installe Docker
curl -fsSL https://get.docker.com | sh
# Autorise votre utilisateur à utiliser Docker (déconnectez-vous puis reconnectez-vous ensuite)
sudo usermod -aG docker "$USER"
# Fait démarrer Docker automatiquement à chaque allumage du serveur
sudo systemctl enable docker
# Vérifie que tout est prêt
docker compose version
```

5.2 Récupérer l'application

```
# Télécharge le code (remplacez l'adresse par celle fournie)
git clone <ADRESSE_DU_DEPOT> erp
cd erp/deploy/agency
```

5.3 Créer et remplir le fichier de réglages `.env`

```
# Copie le modèle fourni vers un vrai fichier .env, puis ouvrez-le pour l'éditer
cp .env.example .env
nano .env
```

Complétez les valeurs (voir la partie 6 pour le détail de chaque ligne) :

```
SECRET_KEY=# une longue suite de caractères au hasard
DJANGO_SETTINGS_MODULE=config.settings.production
DEBUG=0
ALLOWED_HOSTS=nom-du-serveur,localhost,127.0.0.1

POSTGRES_DB=erp_agency
POSTGRES_USER=erp_agency
POSTGRES_PASSWORD=# un mot de passe fort
DATABASE_URL=postgres://erp_agency:<mot de passe>@db:5432/erp_agency
WEB_PORT=8000

SYNC_ENABLED=1
NODE_ROLE=agency
NODE_ID=# un UUID unique (voir l'astuce ci-dessous)
NODE_AGENCY_ID=# l'UUID de l'agence, créé sur le serveur central
SYNC_MASTER_URL=https://<serveur-central>/api/v1
```

```
SYNC_TOKEN=# le même jeton que le serveur central
SYNC_INTERVAL=90
RESERVE_INTERVAL=20
FAKE_PAYMENTS=1
```

Générer un UUID pour `NODE_ID`

```
python3 -c "import uuid; print(uuid.uuid4())"
```

Copiez le résultat dans `NODE_ID`. Dans **nano**, enregistrez avec **Ctrl+O** puis **Entrée**, et quittez avec **Ctrl+X**.

Important

Le `SYNC_TOKEN` doit être **identique** à celui du serveur central, et le `NODE_AGENCY_ID` doit correspondre à une agence **déjà créée** sur le centre.

5.4 Démarrer l'application

```
# Construit et lance tous les services en arrière-plan
docker compose up -d --build
# Vérifie que 4 services tournent : web, db, sync_worker, reserve_worker
docker compose ps
```

L'interface est maintenant accessible dans un navigateur à l'adresse `http://nom-du-serveur:8000`.

5.5 Charger la configuration depuis le serveur central (une fois)

```
docker compose run --rm web python manage.py bootstrap_from_master
```

Cette commande récupère toute la configuration (institutions, agences, tarifs, comptes, droits) : l'agence est alors immédiatement opérationnelle.

5.6 Vérifier que la synchronisation fonctionne

- ✓ Sur le serveur central, l'agence apparaît **« en ligne »** (menu de statut de synchronisation).
- ✓ Une réservation créée en agence **remonte** au centre après 1–2 minutes.
- ✓ Un tarif modifié sur le centre **redescend** en agence après 1–2 minutes.

Voir les journaux (logs)

```
docker compose logs -f web
docker compose logs -f sync_worker
```

(Appuyez sur **Ctrl+C** pour arrêter l'affichage.)

6 Les réglages expliqués

Chaque ligne du fichier `.env` configure un aspect de l'application. Voici les plus importantes.

6.1 Réglages généraux

Réglage	À quoi ça sert
`SECRET_KEY`	Clé de sécurité interne. Mettez une longue suite de caractères au hasard, gardée secrète.
`DEBUG`	`0` en production (obligatoire). `1` uniquement pour du test.
`ALLOWED_HOSTS`	Les adresses par lesquelles on accède au serveur (séparées par des virgules).
`DATABASE_URL`	L'adresse de la base de données. En agence, laissez la valeur fournie (elle pointe vers le service `db`).
`WEB_PORT`	Le « port » web (par défaut `8000`) : l'interface est sur `http://serveur:8000`.

6.2 Réglages de synchronisation (agence ↔ centre)

Réglage	Valeur en agence
`SYNC_ENABLED`	`1` pour activer la synchronisation.
`NODE_ROLE`	`agency` (le centre, lui, utilise `master`).
`NODE_ID`	Un UUID **unique** pour ce serveur d'agence.
`NODE_AGENCY_ID`	L'UUID de l'agence (créée sur le centre) que ce serveur dessert.
`SYNC_MASTER_URL`	L'adresse du centre suivie de `/api/v1`.
`SYNC_TOKEN`	Le mot de passe partagé — **identique** à celui du centre.

6.3 Paiement en ligne (optionnel)

Réglage	À quoi ça sert
`FAKE_PAYMENTS`	`1` = paiements simulés (pour tester). `0` + configuration Campay = vrai Mobile Money.

7 Première utilisation

1. Ouvrez un navigateur et allez sur l'adresse de l'application (le centre, ou `http://serveur:8000` en agence).
2. Connectez-vous avec l'e-mail et le mot de passe temporaire de l'administrateur (fournis à la création de l'institution).
3. Au premier accès, il vous sera demandé de **changer le mot de passe**.
4. Choisissez votre **langue** en haut à droite (français, anglais, espagnol, portugais).
5. Vérifiez la configuration (agences, tarifs, utilisateurs) puis commencez à vendre.

> **Manuels d'utilisation**
> Pour l'usage quotidien (voyages, réservations, colis, trésorerie…), reportez-vous aux **Manuels 01 à 11** fournis avec l'application.

8 Sauvegardes, coupures de courant & entretien

8.1 Reprise après une coupure de courant

Grâce au réglage `restart: unless-stopped` et à Docker activé au démarrage (étape 5.1), l'application **redémarre toute seule** au retour du courant.

> **Fortement recommandé**
>
> Installez un **onduleur (UPS)** sur le serveur et le routeur/box Internet : cela évite les arrêts brutaux et protège la base de données.

8.2 Sauvegarder la base de l'agence (chaque jour)

Créez une sauvegarde automatique quotidienne :

```
# Fichier : /etc/cron.daily/erp-backup  (rendez-le exécutable : chmod +x)
cd /chemin/erp/deploy/agency
docker compose exec -T db pg_dump -U erp_agency erp_agency \
  | gzip > /var/backups/erp_agency_$(date +%F).sql.gz
# Garde 14 jours d'historique
find /var/backups -name 'erp_agency_*.sql.gz' -mtime +14 -delete
```

> **Restaurer une sauvegarde**
>
> ```
> gunzip -c fichier.sql.gz | docker compose exec -T db psql -U erp_agency
> erp_agency
> ```

8.3 Mettre à jour l'application

```
# Récupère la dernière version puis reconstruit
git pull
docker compose up -d --build
```

> **Ordre à respecter**
>
> Mettez toujours à jour le **serveur central en premier**, puis les agences. Sinon la synchronisation se met en pause (écart de version) jusqu'à ce que tout soit à la même version.

9 En cas de problème

Ce que vous constatez	Que faire
Erreur mentionnant « no such column »	Une mise à jour de base n'a pas été appliquée. Relancez : `docker compose run --rm web python manage.py migrate`
La synchronisation renvoie une erreur **409**	Le centre et l'agence n'ont pas la même version. Mettez à jour le centre puis l'agence (même version partout).
« Agence hors ligne » (réservation en ligne refusée)	Le service `reserve_worker` est arrêté, ou `SYNC_MASTER_URL` / `SYNC_TOKEN` sont incorrects. Vérifiez-les, puis `docker compose up -d`.
Erreurs « CORS » / « CSRF » dans le navigateur	L'adresse du frontend n'est pas déclarée. Ajoutez-la dans `CORS_ALLOWED_ORIGINS` et `CSRF_TRUSTED_ORIGINS` (côté serveur central).
Impossible de modifier un tarif/une agence sur le serveur d'agence	C'est normal : la configuration est en lecture seule en agence. Modifiez-la **sur le serveur central** ; elle redescend ensuite.
Une nouvelle permission n'apparaît pas pour un utilisateur	Accordez-la à son rôle, puis demandez-lui de se **déconnecter/reconnecter** (les administrateurs y ont accès d'office).
Les services ne démarrent pas	Regardez les journaux : `docker compose logs -f web`. Vérifiez le fichier `.env` (fautes de frappe, valeurs manquantes).

> **Besoin d'aide ?**
> Conservez à portée de main : l'adresse du serveur central, le fichier `.env` de l'agence (sans le partager publiquement), et la sortie de `docker compose ps` — cela accélère le support.